

ARFL

Privacy in the Dark Cloud

A Decentralised, Privacy-Respecting Bandwidth Marketplace Powered by Bitcoin

arfl.io · github.com/Radi-Labs/ARFL · #arfl on Nostr

Abstract. *The VPN industry sold privacy. What it actually sold was trust in a company — a company with servers that can be seized, a CEO that can be pressured, and invoices that link identity to browsing history. ARFL is a decentralised bandwidth marketplace that removes all three problems from the architecture rather than promising not to abuse them. ARFL composes four open protocols — WireGuard for encrypted transport, Nostr for pseudonymous node discovery and hub-issued node authorization, the Bitcoin Lightning Network for payment, and Cashu ecash for blind-signed bandwidth credentials — into a self-sustaining network that anyone can build on. Node operators plug directly into the protocol, earning Bitcoin from bandwidth they already own. Hubs are products built on top of the protocol: they set their own prices and compete on brand and service, while the protocol itself defines no payment destination of its own and can collect no fee. A user's WireGuard keypair — generated locally and never transmitted — is their sole identity, and a Cashu blind signature severs the link between the sats a user paid and the bandwidth token a node later redeems. ARFL is explicit about the boundaries of what it guarantees: nodes are authorised to route by a hub-issued attestation rather than by trustless collateral, and the hub is the real-time arbiter of double-spend checks. Mathematics enforces what it can; everything else is stated plainly rather than assumed.*

Draft v0.4 · 2026 · No rights reserved

1. Introduction

Commerce on the internet has come to rely almost entirely on centralised intermediaries to provide privacy services. The VPN industry, despite its promises, reproduces the same structural problem: a central company holds the keys, the logs, and the liability. That company can be subpoenaed, hacked, acquired, or pressured. Its users have no recourse and no verification. They are asked to trust. Trust is not privacy.

The subscription model compounds the problem. Monthly payments require credit cards. Credit cards require identity. Users sign up at promotional rates and renew at multiples of the original price — surprised, locked in, and no more private than before. Every centralised VPN, regardless of its privacy claims, can be subpoenaed, hacked, acquired, or pressured into policy changes. The architecture is the vulnerability.

What is needed is a network based on cryptographic proof and economic incentive rather than trust — one where no central server exists to seize, no account exists to subpoena, and no company exists to pressure. In this paper we describe ARFL: a decentralised protocol that aggregates four existing open protocols into a self-sustaining bandwidth marketplace owned by nobody and available to everyone.

ARFL defines two classes of participant. Node operators interact directly with the protocol, contributing bandwidth and earning Bitcoin. Hubs are products built on top of the protocol, providing user-facing experiences, vetting the nodes they route traffic through, and competing on brand and service quality. Between users and node operators sits no intermediary the protocol itself defines — a hub is simply a business built on the protocol, in the same sense a company is a business built on AWS or on Stripe. The protocol has no visibility into, and no opinion on, how a hub makes money.

The key properties of ARFL are: (1) no user accounts or identity — a locally generated WireGuard keypair is the user's sole credential; (2) no subscription — users purchase bandwidth in discrete tiers via Bitcoin Lightning, at a price each hub sets independently; (3) no logs — the architecture makes logging impossible, provable by inspection of open source code; (4) no token — the incentive mechanism is Bitcoin; (5) no protocol fee — the protocol defines no payment address of its own, so it has nothing to collect a cut into; (6) unlinkable bandwidth credentials — Cashu blind signatures mean a hub cannot connect the sats a user paid to the token a node later redeems.

Everything described in this paper is implemented and running. Where a mechanism is planned rather than live, it is labelled as such, and Section 10 sets out plainly what the protocol does and does not protect against. A privacy network that overstates its guarantees is worse than one that states them accurately, because users make real decisions on the strength of those claims.

2. System Architecture

ARFL is composed of four protocol layers and two participant layers. No new cryptography is introduced. The contribution of ARFL is the composition of existing protocols into a coherent privacy system with aligned economic incentives, and a design in which the protocol layer itself has no payment destination to collect a fee into.

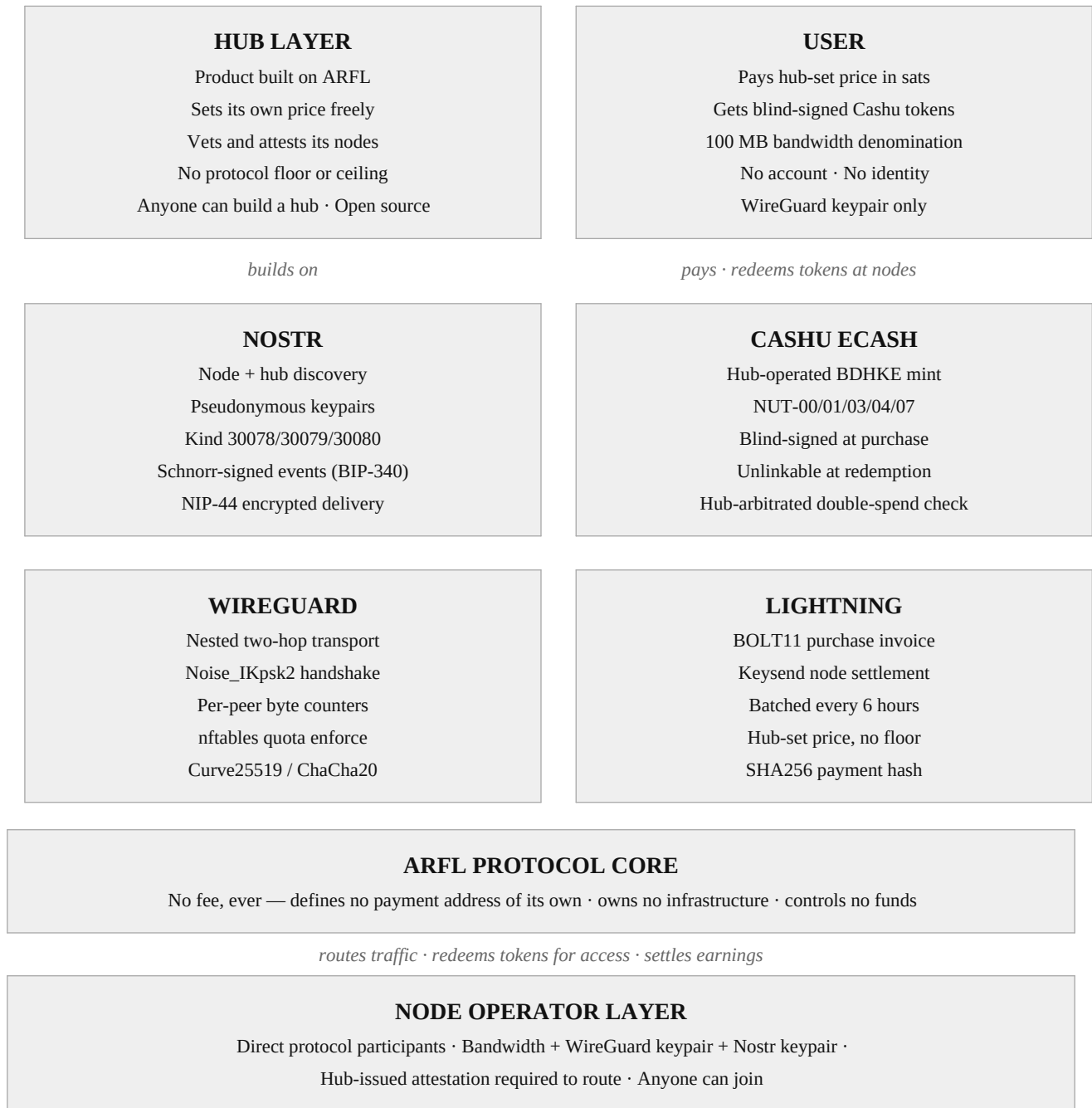
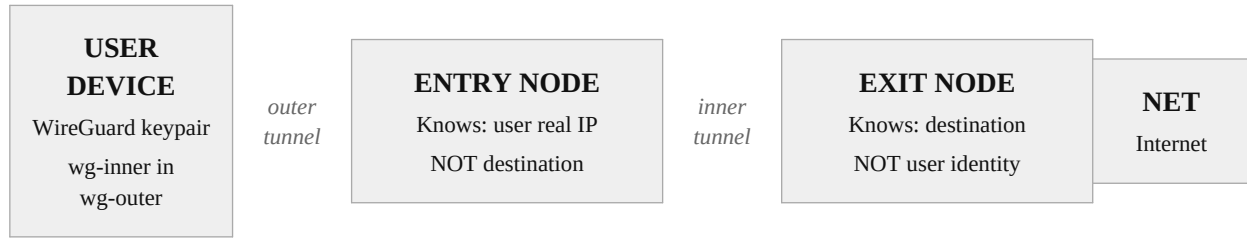


Figure 1. ARFL protocol architecture. The protocol core defines no address of its own; the hub sets price and vets nodes, and node earnings are paid out in full share, less the hub's own configured margin.

The session data flow proceeds through a mandatory two-hop routing design, implemented as **nested** WireGuard tunnels rather than sequential forwarding: the client builds an inner tunnel to the exit node that is carried inside an outer tunnel to the entry node. This matters because a naive forward-only relay would let the entry node decrypt and inspect packets, seeing destination IPs — nesting keeps the entry node blind to everything past an encrypted blob addressed to the exit. No single node holds the complete picture of a user session.



The ARFL client selects and pairs entry and exit nodes. Neither node is informed of the other's identity. All routing decisions are made unilaterally by the client. DNS queries route through the tunnel to Quad9 (9.9.9.9) — no DNS leak possible.

Figure 2. Nested two-hop session routing.

3. Cryptographic Primitives

ARFL inherits its cryptographic security from four independent protocol stacks, each well-audited and widely deployed in production systems.

3.1 WireGuard

WireGuard implements the Noise_IKpsk2 handshake protocol, providing mutual authentication and forward secrecy. The cryptographic primitives are fixed and non-negotiable, eliminating downgrade attacks by design:

- **Curve25519** — Elliptic-curve Diffie-Hellman key exchange.
- **ChaCha20-Poly1305** — Authenticated encryption (AEAD) for all tunnel traffic.
- **BLAKE2s** — Cryptographic hashing for key derivation and MAC operations.
- **SipHash24** — Hashtable keying to prevent denial-of-service via hash collision.

Each WireGuard peer is identified by a 256-bit Curve25519 public key. The ARFL client generates a keypair locally on first launch. This keypair is the user's sole persistent identity — no username, no email, no password.

3.2 Nostr

Nostr uses secp256k1 elliptic curve keypairs for identity. Every node and hub announcement is a Nostr event signed with a BIP-340 Schnorr signature over the SHA256 hash of the canonical event serialisation (NIP-01). Node-to-node token delivery additionally uses NIP-44 encrypted direct messages so that a relay observing the wire never sees the payload. Operators are identified solely by their Nostr public key — no real-world identity is required or stored.

```

Event ID:  SHA256(canonical_json_serialisation)
Signature:  Schnorr(private_key, event_id)  -- BIP-340
Encoding:   NIP-19 bech32 (npub / nsec) for display
Transport:  WebSocket to Nostr relays, HTTP fallback if relays reject
  
```

3.3 Lightning Network

Purchases are made via BOLT11 invoices. Each invoice encodes a SHA256 payment hash derived from a secret preimage known only to the payee; payment is cryptographically proven by revealing the preimage. Node earnings are paid out via Lightning keysend, batched every six hours by the hub's settlement engine. The entire ARFL economy is denominated in satoshis. USD figures anywhere in this document are human-readable references only and carry no protocol obligation.

3.4 Cashu Ecash (Blind Diffie-Hellman Key Exchange)

Bandwidth credentials are issued by a Cashu-compatible ecash mint operated by the hub. The mint implements Blind Diffie-Hellman Key Exchange (BDHKE) over secp256k1, following the Cashu NUT specification (NUT-00 through NUT-07): the client blinds a set of messages before sending them to the hub for signing, the hub signs the blinded messages without seeing their contents, and the client unblinds the resulting signatures locally to produce spendable tokens.

The privacy guarantee this buys is specific and worth stating precisely: when a node later redeems a token with the hub, the hub cannot mathematically link that redemption back to the Lightning payment that originally minted it. This is what makes a bandwidth token unlinkable to the purchase that paid for it — the same construction Chaumian ecash has used since 1983, applied here to bandwidth credentials denominated in satoshis rather than to a standalone currency.

The limits of that guarantee matter as much as the guarantee itself. BDHKE unlinks a token from its purchase. It does not, by itself, prevent a hub from correlating session timing across the two hops whose peer provisioning it also mediates, and it places no collateral behind a node's behaviour. Section 10.1 sets out these limits in full.

Double-spend protection is enforced by the hub, which is the single source of truth for whether a given proof has already been redeemed (NUT-07 `/checkstate`, and the ARFL-specific `/v1/redeem` a node calls before granting tunnel access). This makes the hub a real-time arbiter for double-spend detection — a node cannot verify a token offline. Section 10.1 discusses this trade-off explicitly.

4. Node Discovery and Authorization

Node operators announce availability by publishing signed Nostr events to public relays; the ARFL client subscribes to these relays and maintains a live index of available nodes. But announcing is not the same as being trusted to route: a node must additionally hold a hub-issued attestation before any compliant client or hub will provision it with client peers. This section describes both halves — the permissionless discovery layer and the permissioned authorization layer sitting on top of it.

4.1 Node Announcement Event

ARFL defines a custom Nostr event kind 30078 in the parameterised replaceable event range (30000–39999 per NIP-01), allowing relays to replace stale announcements by the same pubkey and 'd' tag. Node software republishes this event every 60 seconds. When public relays rate-limit high-frequency publishing, the node falls back to a direct POST /announce call to the hub, which validates the same signature and attestation a relay-delivered event would carry.

```
{
  "kind": 30078,
  "pubkey": "<node_nostr_pubkey_hex>",
  "created_at": <unix_timestamp>,
  "tags": [
    ["d", "<unique_node_id>"],
    ["wg_pubkey", "<wireguard_pubkey_base64>"],
    ["endpoint", "<ip_address>:<port>"],
    ["connect_url", "<http_base_url_for_token_delivery>"],
    ["lnurl", "<lnurl_pay_address>"],
    ["deposit_sats", "<self_reported, not enforced in v1 -- Sec. 7>"],
    ["upload", "<mbps>"], ["download", "<mbps>"],
    ["load", "<current_user_count>"], ["capacity", "<max_user_count>"],
    ["role", "both|entry|exit"], ["version", "<arfl_node_version>"]
  ],
  "content": "",
  "id": "<sha256_of_canonical_serialisation>",
  "sig": "<schnorr_signature>"
}
```

4.2 Hub-Issued Node Attestation

A signed Nostr announcement proves a node controls a keypair; it does not prove the hub has vetted that node. ARFL closes this gap with a structured **attestation** — a signed voucher from the hub, conceptually similar to a TLS certificate: it does not merely assert 'this node exists,' it asserts 'this specific WireGuard key, under this operator identity, in these roles, is authorized until this timestamp.' Change any bound field and the attestation no longer applies.

```
type Attestation struct {
    Protocol      string    // "arfl-node-attestation-v1"
    HubPubkey     string    // hub Nostr pubkey (issuer)
    NodePubkey    string    // node Nostr pubkey (subject)
    NodeWGPubkey  string    // bound WireGuard key
    OperatorID    string    // hub-assigned operator identity
    AllowedRoles  []string  // ["entry"], ["exit"], or both
    IssuedAt      int64
    ExpiresAt     int64     // issued_at + 6 hours
    AttestationID string    // SHA-256 over the above
}
```

```

    Signature    string    // hub BIP-340 Schnorr sig over AttestationID
}

```

The 6-hour attestation TTL is deliberately short: if a node is compromised, the exposure window is bounded to a few hours rather than persisting indefinitely. Underneath the attestation sits a longer-lived **lease** — a record in the hub's database (default 90 days) that governs whether a node is *allowed* to refresh its attestation at all. Attestations are the session key that rotates frequently; leases are the access policy that governs who can keep refreshing one. An operator can be revoked instantly by flipping the lease's **revoked** flag — the next refresh attempt is rejected regardless of how much time remains on the current attestation.

This is a real centralisation trade-off and is stated plainly: a node cannot route traffic for a hub that has not vetted it, and a hub can revoke a node unilaterally. Attestation is what v1 uses in place of the trustless, deposit-backed authorisation described in Section 7, which is specified but not yet enforced. Discovery remains permissionless — anyone can announce a node on Nostr and any hub may attest it — but routing for a given hub is that hub's decision.

4.3 Node Selection Algorithm

The client selects an entry and exit node pair on each new session from among nodes carrying a valid, unexpired attestation:

1. Discard any node without a valid, unexpired hub attestation bound to its announced WireGuard key.
2. Filter: status=online, load < capacity, version compatible.
3. Score each node:
 - score = (capacity - load) / capacity
 - * upload_mbps / 50
 - * uptime_ratio
4. Select entry node: highest score, role=entry|both.
5. Select exit node: highest score, role=exit|both, different operator pubkey to entry.
6. Pairing established client-side only. Neither node is informed of the other.

4.4 Collusion Prevention

The two-node design prevents collusion by construction. Entry and exit nodes are selected independently and identified only by pseudonymous Nostr keypairs; they never communicate directly, and all routing decisions are made unilaterally by the client. An operator running multiple nodes under different keypairs cannot coordinate a fraudulent session because the nodes have no awareness of each other's involvement. Cashu's blind signatures reinforce this at the payment layer: the hub cannot link a token's mint operation to its later redemption, so even a hub attempting to correlate sessions cannot do so by payment timestamp alone (though see the timing-correlation caveat in Section 10.1).

5. VPN Transport via Nested WireGuard

WireGuard provides the encrypted transport layer. ARFL node software manages WireGuard programmatically using the wgctrl library (golang.zx2c4.com/wireguard/wgctrl, MIT licensed), which communicates with the WireGuard kernel module via generic netlink. The CLI is never used in production.

5.1 Session Establishment

1. Client generates or loads its WireGuard keypair.
2. Client reads Nostr, selects an attested entry + exit pair (Sec. 4.3).
3. Client mints/redeems Cashu tokens against the hub (Sec. 6).
4. Client delivers its token share to each node -- today, a direct HTTPS POST to the node's /cashu-connect endpoint, made BEFORE the tunnel is up. NIP-44 encrypted delivery over Nostr is implemented (internal/client/token_sender.go) but not yet wired into the client by default -- see the caveat in Sec. 10.1.
5. Node verifies the token with the hub via POST /v1/redeem.
6. Node configures the client as a WireGuard peer via wgctrl.
7. Client builds wg-outer (to entry) and wg-inner (to exit, nested inside wg-outer).
8. Noise_IKpsk2 handshakes complete on both tunnels.
9. Routes set: exit IP via outer tunnel, 0.0.0.0/1 + 128.0.0.0/1 via inner tunnel, entry IP via the default gateway (untunnelled).
10. DNS set to 9.9.9.9 (Quad9) inside the tunnel.
11. Traffic flows: app -> wg-inner -> wg-outer -> entry -> exit -> internet.

Tunnels route IPv4 only (0.0.0.0/1 and 128.0.0.0/1, the split-default-route trick, rather than 0.0.0.0/0). IPv6 traffic is not encapsulated at all in v1 — a client must disable IPv6 at the OS level, or it leaks around the tunnel entirely rather than merely being unprotected.

5.2 Per-Peer Byte Accounting

WireGuard maintains monotonic 64-bit byte counters per peer, updated per-packet in the kernel data path. The node daemon polls these counters every five seconds via wgctrl. Nodes sign and submit usage reports built from these counters; the hub's settlement engine computes **billable bytes as the minimum of what the entry node reported and what the exit node reported** for a given session — one rule, no adjustments, no human overrides.

5.3 Quota Enforcement

Quota is enforced in 256 MB kernel slabs via nftables, keyed by client tunnel IP on the exit node (where cleartext traffic is visible and measurable). As each slab is consumed the node refreshes the quota element against the client's remaining token balance; when the balance reaches zero, the peer is removed via wgctrl and nftables drops further packets unconditionally. This two-layer approach — wgctrl counters for accounting, nftables quotas for enforcement — eliminates the polling-overshoot window a purely software cutoff would leave open.

5.4 Key Management and Device Portability

The user's WireGuard private key is generated locally on first launch. It is never transmitted to any ARFL component. To migrate to a new device, the user exports the private key (e.g. as a QR code) from the original device and imports it on the new one. Unredeemed Cashu tokens are bearer instruments stored client-side — whoever holds the token secret can spend it, so losing the device without a backup means losing the remaining balance. This is an intentional tradeoff: the same property that removes account recovery removes identity linkage.

6. Payment Architecture

ARFL sells bandwidth in discrete, prepaid tiers rather than as a 250 GB flat bundle at a single protocol-fixed price. **Pricing is entirely a hub decision.** The protocol has no opinion on it, enforces no floor or ceiling, and nothing below should be read as a suggested or default price — each hub operator sets whatever it charges for whatever tiers it chooses to offer. The table below reproduces the reference implementation's example configuration purely to illustrate the mechanics (tier → token denomination), not as a protocol or project-recommended price list.

Tier	Bytes	Example Price	Token Denomination
1 GB	1,000,000,000	500 sats*	10 × 100 MB tokens
10 GB	10,000,000,000	4,000 sats*	100 × 100 MB tokens
50 GB	50,000,000,000	15,000 sats*	500 × 100 MB tokens

Table 1. Worked example only (), taken from the reference implementation's sample config — not a protocol default, a price floor, or a recommendation. Every hub operator sets its own tiers and prices. A tier is atomic per 100 MB token: partial consumption of a token forfeits the remainder, trading at most 100 MB of waste per redemption for a much simpler accounting model and a clean fit with blind-signature denominations.*

6.1 Protocol and Configuration Values

Parameter	Value	Rationale
Token denomination	100 MB, tunable	Atomic unit; configurable without code changes
Hub margin	20% default, 0–50% configurable	Hub's own retained share, not a protocol fee
Routing split	50 / 50 of the post-margin remainder (entry / exit)	Even split between the two hops
Bundle price	Set freely by each hub operator, per tier	No protocol-enforced floor or ceiling — not the protocol's decision
Settlement interval	6 hours, configurable	Batched Lightning keysend payout to nodes
Minimum payout	1,000 sats	Avoids dust-sized keysend payments
Attestation TTL	6 hours	Bounds exposure from a compromised node
Node lease window	90 days default	Longer-lived authorization policy, revocable anytime
Min bandwidth	50 Mbps up/down	Baseline service quality for node operators
Ping interval	60 seconds	Node health monitoring
DNS resolver	9.9.9.9 (Quad9)	Privacy-preserving resolver, enforced in-tunnel
Protocol fee	0%, always	The protocol defines no address of its own to collect into

Table 2. Configuration values in v1. With the exception of the attestation TTL and the absence of a protocol fee address, which are structural, these are hub configuration defaults rather than protocol-enforced constants.

6.2 Purchase and Mint Flow

The complete purchase flow, matching the hub's Cashu-compatible mint endpoints (NUT-04):

1. Client selects a tier; hub returns its /info with current tiers and pricing.
2. `POST /v1/mint/quote/bolt11 {amount, unit: 'sat'}`
-> hub returns a BOLT11 invoice and a quote id.
3. User pays the invoice from any Lightning wallet.
4. Hub's settlement listener marks the quote PAID.
5. Client generates blinded messages locally and calls
`POST /v1/mint/bolt11 {quote, outputs: [blinded_messages]}`.
6. Hub blind-signs the messages -- `POST /v1/mint/bolt11` returns blind signatures. The hub never sees the unblinded secrets.
7. Client unblinds the signatures locally, producing spendable Cashu proofs (100 MB tokens). This is the point at which the purchase becomes cryptographically unlinkable to later use.
8. Tokens are stored on the client device, not on the hub.

6.3 Token Delivery and Redemption

To use a token, the client must get it to the entry and exit nodes so they can verify it with the hub and grant tunnel access. Two delivery paths exist in the codebase today:

- **Default path:** the client makes a direct HTTPS call to each node's `/cashu-connect` endpoint, before the WireGuard tunnel is established. This is simple and works, but it means each node sees the client's real IP address during setup — a privacy cost the token's blind-signature unlinkability does not cover.
- **Implemented, not yet the default:** NIP-44 encrypted Nostr direct messages (`internal/client/token_sender.go`) deliver the token to each node's Nostr pubkey without ever touching the client's IP. This closes the setup-time IP exposure above, and is the intended path once wired into the client by default.

Either way, the node presents the proofs it received to the hub via `POST /v1/redeem`. The hub checks the proof against its spent-set (NUT-07), rejects any duplicate, and returns the bytes the node may allow. This makes the hub the real-time arbiter of double-spend detection: a node cannot verify a token without asking the hub. That is the cost of a single-mint design, and Section 15 sets out what a federated mint would change.

6.4 Node Earnings Settlement

Token redemption grants tunnel access; it is separate from how node *operators* get paid for the bandwidth they served. The hub's settlement engine runs on a fixed interval (default six hours): it aggregates signed usage reports into billable bytes per node (Section 5.2), deducts the hub's configured margin, splits the remainder 50/50 between the entry and exit node that served each session, and pays each node's registered Lightning address via keysend. Settlement is idempotent, keyed by (period, node), so a hub crash mid-run can restart safely without double-paying. Every payout carries an explicit state — `pending` → `paid`, or `pending` → `failed` → `retrying`, capped at three retries before a payout is flagged for manual review — so a failed keysend never leaves sats silently lost or double-counted.

The trade-off in batched settlement is explicit: the hub holds settlement funds for up to the settlement interval, and node payment amounts are visible to the hub, since the hub computes them. What the hub cannot do is link a specific token redemption back to the Lightning payment that funded it — that guarantee lives entirely in the BDHKE construction (Section 3.4) and does not depend on how node earnings are later paid out.

6.5 Wallet Sovereignty

ARFL places no requirements on the type of Lightning wallet used by users or node operators. Custodial, non-custodial, KYC and KYC-free wallets are all compatible — any wallet capable of paying a BOLT11 invoice and receiving a keysend payment is sufficient. Privacy and custody decisions are the sole responsibility of the wallet holder. ARFL is a protocol, not a custodian.

7. Node Authorization, Sybil Resistance, and Deposits

v1 does not enforce a node security deposit. There is no deposit-gated capacity cap and no automated user refund funded from a node's own collateral. This is a deliberate v1 choice, and it is stated up front because it bounds the Sybil analysis in Section 10.2.

The protocol package defines the minimum deposit as a named constant, reserved for the mechanism described below:

```
// MinNodeDepositSats is the minimum deposit for node attestation.  
// Not enforced in v1, to maximise node onboarding.  
// Enforced from v2 onwards as a Sybil-resistance measure.  
const MinNodeDepositSats = 50000
```

What v1 uses instead is hub-issued attestation and leases (Section 4.2): a node cannot route traffic without a hub's active, unexpired sign-off, and that sign-off can be revoked immediately. This is a permissioned but far simpler onboarding path than bonded collateral, chosen so that node operators can join a young network without first locking up capital in a mechanism whose parameters are still being calibrated against real traffic.

Concretely: an attacker seeking to control a large share of the node network does not need capital to do it. They need attestations issued to them, which is an operational control exercised by each hub over who it vets, not a cryptoeconomic one. Section 10.2 sets out the resulting Sybil posture.

The v2 mechanism uses `MinNodeDepositSats` as a bonded requirement: a node's advertised capacity is capped as a function of its deposit, and automated user refunds are funded from that collateral when a node goes dark. Custody of that collateral is the open design question — a neutral federation holding it, rather than any single hub, would additionally allow tokens minted by one hub to be redeemed by nodes serving another hub's traffic, which a single-mint design cannot offer. Section 15 covers this.

Until that lands, node reliability is enforced by delisting rather than by financial penalty: a node that goes offline stops having its attestation renewed, or has its lease revoked. The deposit-related webhook events in Section 8 are defined against the v2 mechanism and carry no data in v1.

8. Webhook Architecture

The hub can push real-time events to subscribed hubs (in multi-hub deployments) or downstream integrations via encrypted Nostr direct messages (NIP-04) or a direct HTTPS webhook, specified by publishing a kind 30080 Nostr event.

Event	Trigger	Typical Action
NODE_OFFLINE	Node fails scheduled ping	Start offline timer
NODE_ONLINE	Node resumes ping	Cancel timer if active
DEPOSIT_LOW	Reserved for v2 deposit model	Not populated in v1
DEPOSIT_DEPLETED	Reserved for v2 deposit model	Not populated in v1
GB_BALANCE_LOW	Client token balance below threshold	Push purchase prompt
SESSION_STARTED	WireGuard tunnel established	Log session start
SESSION_ENDED	Tunnel closed	Update GB accounting
SETTLEMENT_SENT	Settlement payout executed	Reconcile node ledger

Table 3. ARFL webhook event types. Deposit-related events are defined in the type system ahead of the v2 deposit mechanism (Section 7) but carry no data in the current release.

9. Privacy Architecture

ARFL is designed so that no single entity accumulates sufficient information to reconstruct a user session.

Component	Knows	Does Not Know
Entry node	User's real source IP; encrypted inner-tunnel blob	Destination, user identity
Exit node	Traffic destination; client tunnel IP	User's real IP, user identity
Hub / mint	WireGuard pubkey; mint amounts; redeemed proofs	Which purchase funded which redemption; browsing
Nostr relay	Node and hub announcements	User activity, sessions
Cashu mint keys	Total value minted and redeemed	Mint-to-redeem link (BDHKE)
Anyone	Nothing complete	Full session picture

Table 4. Information compartmentalisation across ARFL components (v1). The hub and the mint are a single operator role, so the "does not know" column for that row rests entirely on the BDHKE construction rather than on organisational separation between them.

DNS queries are routed through the encrypted WireGuard tunnel to Quad9 (9.9.9.9), enforced in the WireGuard interface configuration delivered at session start. The exit node never sees DNS queries in cleartext outside the tunnel, and no DNS leak is architecturally possible — provided IPv6 is disabled at the OS level (Section 5.1). Because the hub's code is open source, its stated data-handling claims are verifiable by inspection, not merely

asserted in a policy document.

10. Security Model

10.1 Honest Threat Model

ARFL is positioned as a **privacy-respecting bandwidth marketplace**, not as an untraceable VPN — a deliberate choice by the project (recorded in its decision log as "product positioning") because the stronger framing carries legal and perception risk the architecture does not actually back up everywhere. ARFL protects against:

- ISP traffic surveillance and deep packet inspection.
- Network-level IP address tracking and logging.
- DNS query surveillance and domain-level logging.
- Identity-linked browsing history.
- Legal compulsion of a central VPN provider.
- Server seizure by authorities.
- Corporate acquisition or policy change.
- Linking a bandwidth token to the Lightning payment that funded it (BDHKE, Section 3.4).

ARFL does *not* protect against, and this list is treated as load-bearing rather than as boilerplate:

- Malware or compromise of the user's own device.
- Browser fingerprinting techniques.
- Services the user voluntarily authenticates into.
- A coordinated adversary controlling a majority of the node network simultaneously.
- **Double-spend detection without the hub.** The hub is the real-time arbiter for every token redemption (Section 6.3); a node cannot verify a proof offline.
- **Setup-time IP exposure to nodes.** The client's default path delivers tokens via a direct HTTPS call before the tunnel is up, so both nodes see the client's real IP during session setup (Section 6.3). The NIP-44 path that would close this exists but is not yet the default.
- **Hub timing correlation.** The hub cannot link a redemption to its funding payment, but a compromised hub mediating peer provisioning for both hops could potentially correlate a session by timing alone. Client-side node pairing sourced purely from the Nostr relay index, without hub mediation of provisioning, is listed as future work for exactly this reason.
- **IPv6 leakage.** Tunnels encapsulate IPv4 only; IPv6 must be disabled at the OS level or it bypasses the tunnel entirely (Section 5.1).

- **Cloud-hosted node economics.** The residential, flat-rate bandwidth model (Section 12) does not work for metered cloud hosting at current pricing — this is a market-fit boundary, not a security one, but is included here because it bounds who can realistically run a well-behaved node.

10.2 STRIDE Threat-Model Testing

Rather than asserting security properties, the codebase carries dedicated automated tests mapped to the STRIDE threat categories, run under Go's race detector on every change:

STRIDE Category	Server-side	Client-side	Total
Spoofing	2	3	5
Tampering	2	5	7
Repudiation	2	1	3
Information Disclosure	1	4	5
Denial of Service	2	3	5
Elevation of Privilege	3	2	5
Concurrent / Race	2	2	4
Total	14	20	34

Table 5. STRIDE-categorised security tests, part of a 408-test suite covering the full *purchase → pay → redeem → verify → spend → double-spend-detection → unlinkability-proof path end to end*.

10.3 Sybil Resistance

As stated in Section 7, v1 does not enforce the node deposit that would make Sybil resistance a capital-cost argument. The analysis below describes the v2 posture, once `MinNodeDepositSats` is enforced; in v1 the equivalent defence rests on hub-operational vetting instead (Section 4.2).

At the planned 50,000-sat minimum, an attacker seeking to control 1,000 node identities would need to fund 50,000,000 sats in capital, all independently verifiable and all held at risk of depletion through refund obligations once the deposit-and-refund mechanism exists. A node capacity cap tied to deposit size (advertised capacity never exceeding a function of deposit) is the intended complement, bounding how much concurrent-refund liability a single Sybil identity could create.

10.4 Replay Attack Prevention

Each session request includes a token derived from the user's WireGuard public key, the current timestamp, and a random nonce: `token = BLAKE2s(wg_pubkey || timestamp || nonce)`. The hub maintains a short-lived token cache and rejects duplicate tokens within a five-minute window, preventing replay of valid session requests.

10.5 WireGuard Traffic Analysis Resistance

WireGuard is silent by design: it does not respond to unauthenticated packets and sends no response to packets that fail authentication. This makes ARFL nodes invisible to port scanners and resistant to active probing. The fixed-suite cryptography eliminates cipher negotiation as an attack surface.

11. Trust Model

Each security property below is enforced by cryptographic guarantees, economic incentive, operational policy, or open-source verifiability — and the table is explicit about which kind of enforcement backs which property, since they are not interchangeable.

Property	Enforcement
Encrypted transport	WireGuard ChaCha20-Poly1305 — cryptographic
No user identity	WireGuard keypair, locally generated, never transmitted — cryptographic
No session logs	Architecture — no single component sees a full session
Token-to-purchase unlinkability	Cashu BDHKE blind signatures — cryptographic
Node authenticity	Nostr Schnorr signature — cryptographic
Node authorization to route	Hub-issued attestation + lease — operational, revocable, not cryptoeconomic in v1
Double-spend prevention	Hub-held spent-proof set — requires trusting hub availability and honesty
Quota enforcement	nftables kernel quota — OS enforced
Payment proof	BOLT11 preimage — cryptographic
Node payout integrity	Append-only settlement ledger, idempotent per period — software-enforced, hub-operated
No protocol fee	No protocol-defined payment destination exists to collect into — structural
No DNS leak (IPv4)	WireGuard DNS config — architectural
Hub compliance	Open source + STRIDE test suite + community audit

Table 6. Trust properties and enforcement mechanisms in v1.

The hub's trust footprint is real but bounded: it cannot access user browsing activity, cannot deanonymise a session by inspecting a token, and its source code is open and auditable. It is, however, a single point of operational trust for node vetting and double-spend arbitration, and that dependency is stated here directly rather than left for a reader to infer from the architecture.

12. Node Operator Economics

ARFL node operators earn Bitcoin passively by contributing bandwidth to the network. The economic model is analogous to Bitcoin proof-of-work: operators contribute a resource, are rewarded proportionally to contribution, and earn nothing when offline. The Airbnb parallel is also instructive — node operators list bandwidth as hosts list properties; they do not receive free VPN access any more than an Airbnb host receives free stays elsewhere. Users and node operators are entirely separate participant classes.

Node earnings are entirely a function of whatever price and tier mix the hub a node connects through actually chooses — there is no protocol-fixed number to cite, and the figures below are a worked example, not a forecast or a promise. To make the mechanics concrete, the example uses the reference implementation's sample pricing from Table 1 (itself just an illustration, not a recommendation): at a hypothetical 1 GB tier priced at 500 sats with a 20% hub margin, the post-margin 400 sats split 50/50 would leave each routing node 200 sats per GB it relays; a hub charging more or less, or splitting tiers differently, changes every number below proportionally. All figures are given in satoshis rather than pegged to a USD rate that drifts.

Node Activity	Simultaneous Users	GB Served / Month	Example Monthly Earnings*
Quiet	~10	~2 TB	~320,000–400,000 sats
Active	~50	~10 TB	~1.6M–2.0M sats
Busy	~100	~20 TB	~3.2M–4.0M sats

Table 7. Worked example only (), built on Table 1's sample pricing — not a projection or a guaranteed rate. A real hub setting different prices, a different margin, or different tiers changes every figure here; this table exists to show how the mechanics scale with traffic, not what any operator should expect to earn.*

Two-hop routing does not double any single node's bandwidth cost, even though it doubles aggregate network bandwidth for a given purchase: each individual node, entry or exit, relays that session's traffic exactly once, identical to what a single-hop node would bear. The even split across routing roles is fair compensation for being one privacy hop, not a subsidy for an inflated cost that does not exist at the node level.

The project's own honest threat model is blunt about the resulting economics: at these sats-per-GB rates, the model works for residential and unmetered flat-rate connections — operators earning from bandwidth they already own — and is explicitly not viable for commercial cloud hosting, where metered egress cost would exceed the payout. This is treated as a targeting decision, not a defect: the bootstrap problem of low early traffic is a growth problem, not an economics one.

Minimum requirements to run a node in v1:

- 50 Mbps upload and download minimum.
- Any Lightning wallet capable of receiving a keysend payment — no full Lightning node required.
- A hub willing to vet and attest the node (Section 4.2) — no security deposit required in v1.
- 80% uptime commitment.

- ARFL node software (open source, Go).
-

13. Hub Economics

Hub revenue is out of protocol scope by design. The protocol's only stance is structural: it defines no payment address of its own, so there is no mechanism by which it could size, verify, or enforce hub monetisation even if one were wanted. This mirrors the AWS or Stripe relationship — the infrastructure layer has no say in how a business built on top of it prices its own product.

In the reference hub implementation, this takes the concrete form of a configurable `hub_margin_pct` (default 20%, settable 0–50%) retained from each purchase before the remainder splits 50/50 to the routing nodes. A hub is free to run at a thinner margin, subsidise it from elsewhere, or monetise entirely outside the BOLT11 purchase flow — none of that is observed or governed by the protocol layer.

Hubs compete on brand, reliability, and service quality rather than on the underlying node network, since all compliant hubs can draw on the same node pool. Differentiation comes from the experience built on top of the protocol: how quickly a hub responds to node failures, how clearly it discloses its margin and threat model, and the trust its brand commands in the community.

14. Why No Token

Every prior decentralised privacy network introduced a native token: Mysterium (MYST), Sentinel (DVPN), Orchid (OXT). Tokens introduce speculation over utility, regulatory uncertainty across jurisdictions, participation barriers for users unfamiliar with token acquisition, and protocol complexity that distracts from the underlying mission.

ARFL uses Bitcoin — the monetary network that already exists, already works, and is already held by the community most aligned with ARFL's values. The Lightning Network provides the micropayment infrastructure that makes per-tier pricing practical at negligible transaction cost. Cashu is not a competing asset or a new token: it is a bearer-note *format* for bandwidth credentials, denominated entirely in satoshis and redeemable only for bandwidth against the issuing hub — it introduces no new unit of account. No new token is introduced. No ICO is conducted. No token treasury exists. The incentive mechanism is Bitcoin, and Bitcoin alone.

15. Engineering Maturity and Roadmap

ARFL is a working reference implementation, not only a specification. The codebase carries 408 tests across ten packages, all passing with Go's race detector enabled, including the 34 STRIDE-categorised tests in Section 10.2 and an end-to-end integration suite exercising the full purchase → pay → redeem → verify → spend → double-spend-detection → unlinkability-proof path with no external infrastructure required. Architectural decisions are recorded in a public decision log rather than left implicit, and a three-server deployment (hub, entry node, exit node) runs the protocol in production configuration.

Phase	Status	Scope
1–3	Live	WireGuard nested-tunnel transport; Nostr discovery; Lightning purchases
4–6	Live	Blind-signed credentials; client SDK; token→connect flow, LND adapter, Docker testnet
9–12	Live	Client-side node selection + Cashu redemption; node-side gate; NIP-44 delivery (implemented, not default); full E2E test
13–14	Live	Three-server deployment; performance engineering (benchmarks, load test)
15	Planned	LNbits extension (wallet integration)
16	Planned	Mobile app (gomobile bindings)
17	Planned	Multi-hop routing (more than two hops)
v2	Specified	Bonded node deposits, deposit-linked capacity caps, collateral-funded refunds (Section 7)
—	Open design question	Neutral federated mint custody and cross-hub token portability — see below

Table 8. Development status. "Live" phases are exercised by the test suite described above and running in the reference deployment; "planned" phases are not yet started.

On federated custody: a single-mint design cannot make bandwidth credentials portable across hubs, because Cashu tokens are valid only at the mint that issued them. A node serving one hub's traffic therefore cannot redeem a token a user bought from a different hub. Closing that gap means moving mint custody to a neutral federation that no single hub operates — which in turn requires answering who holds guardian keys and how those holders stay operationally independent of every hub in the network, since a hub that also held guardian keys could in principle recognise its own notes at redemption and undo the unlinkability the design depends on. v1 uses a single mint because it matches a trust model where the hub is already the party a user and its nodes deal with directly, and because a working network should not wait on federation governance to exist. This remains an open design question rather than a scheduled deliverable, and it is stated here so that anyone building on ARFL can plan around it.

16. Conclusion

We have described a decentralised bandwidth marketplace that minimises — rather than eliminates outright — the trust a user must place in any single party. By composing WireGuard for encrypted transport, Nostr for discovery and node authorization, the Bitcoin Lightning Network for payment, and a Cashu ecash mint for blind-signed bandwidth credentials, ARFL creates a network where node operators are rewarded in proportion to what they route, users pay only for what they use, and hubs compete on price and service rather than on infrastructure they'd otherwise have to build themselves.

The protocol collects no fee, in this version or any other, because it defines no payment destination of its own to collect one into. The only sats an ARFL-connected hub ever touches are the price it set and disclosed, split with the node operators who did the work of routing the session. Any future funding for ARFL's stewards has to be solved at the business layer — running an ordinary compliant hub, adjacent tooling, grants — never by giving the protocol layer a revenue mechanism of its own.

The boundaries of what v1 guarantees are stated plainly rather than left to inference: nodes are authorised to route by a hub-issued attestation rather than by bonded collateral, the hub is the real-time arbiter for double-spend checks, and the client's default token-delivery path exposes a user's IP to the nodes it selects during setup. None of these are hidden behind the strength of the Cashu blind signature, which solves a real and specific problem — purchase-to-usage unlinkability — without solving every problem a fully trust-minimised design would need to.

The user's WireGuard keypair — generated locally, never transmitted — remains their sole identity. Node operators need only bandwidth and a Lightning wallet. Users need only satoshis and the ARFL application. Privacy is not a product. It is a right this protocol moves closer to, one honestly-scoped release at a time.

References

- [1] D.J. Bernstein. “Curve25519: New Diffie-Hellman Speed Records.” PKC 2006. <https://cr.yp.to/ecdh/curve25519-20060209.pdf>
- [2] Y. Nir, A. Langley. “ChaCha20 and Poly1305 for IETF Protocols.” RFC 8439. 2018. <https://datatracker.ietf.org/doc/html/rfc8439>
- [3] Z. Donenfeld. “WireGuard: Next Generation Kernel Network Tunnel.” NDSS 2017. <https://www.wireguard.com/papers/wireguard.pdf>
- [4] WireGuard control library. wgctrl-go. MIT License. <https://github.com/WireGuard/wgctrl-go>
- [5] fiatjaf et al. “Nostr Protocol — NIP-01: Basic protocol flow.” <https://github.com/nostr-protocol/nostr/blob/master/01.md>
- [6] fiatjaf et al. “Nostr Protocol — NIP-04: Encrypted Direct Messages.” <https://github.com/nostr-protocol/nostr/blob/master/04.md>
- [7] fiatjaf et al. “Nostr Protocol — NIP-44: Encrypted Payloads (Versioned).” <https://github.com/nostr-protocol/nips/blob/master/44.md>
- [8] fiatjaf et al. “Nostr Protocol — NIP-19: bech32-encoded entities.” <https://github.com/nostr-protocol/nostr/blob/master/19.md>
- [9] J. Poon, T. Dryja. “The Bitcoin Lightning Network.” <https://lightning.network/lightning-network-paper.pdf>, 2016.
- [10] Lightning Network Specification. BOLT11: Invoice Protocol. <https://github.com/lightning/bolts/blob/master/11-payment-encoding.md>
- [11] LNURL Specification. <https://github.com/lnurl/luds>
- [12] Cashu. Cashu Protocol Specification (NUTs). <https://github.com/cashubtc/nuts>
- [13] D. Chaum. “Blind Signatures for Untraceable Payments.” CRYPTO ’82, 1983.
- [14] T. Perrin. “The Noise Protocol Framework.” <https://noiseprotocol.org/noise.html>, 2018.
- [15] S. Nakamoto. “Bitcoin: A Peer-to-Peer Electronic Cash System.” <https://bitcoin.org/bitcoin.pdf>, 2008.
- [16] Quad9. Privacy-preserving DNS resolver. <https://www.quad9.net>
- [17] Mysterium Network. Decentralised VPN Protocol. <https://github.com/mysteriumnetwork/node>
- [18] Sentinel. dVPN Protocol. <https://github.com/sentinel-official>
- [19] BIP-340. Schnorr Signatures for secp256k1. <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>
- [20] Fedimint. Federated Chaumian Mints. <https://github.com/fedimint/fedimint>
- [21] Radi Labs. ARFL reference implementation, decision log, and API specification. <https://github.com/Radi-Labs/ARFL>